

FISA DE LUCRU. SUBPROGRAME IN PYTHON

Disciplina: Informatica | **Clasa:** a IX-a | **Limbaj utilizat:** Python

Continuturi vizate	Subprograme: antet, corp, variabile locale/globale, parametri, return. Functii matematice predefinite (abs, round, int, math.sqrt, min, max).
Activitati vizate	Construirea de subprograme proprii, compararea codului modular cu cel nemodular, aplicarea functiilor predefinite pentru calcule, testarea cazurilor limita.
Obiective	Elevul defineste si apeleaza subprograme, distinge intre variabile locale si globale, utilizeaza functii de conversie si rotunjire si implementeaza un mini-proiect.
Durata recomandata	50 minute

1. Teoria

Un **subprogram** (sau o functie) este un bloc de cod reutilizabil, care rezolva o sarcina specifica. Modularizarea codului inseamna impartirea unui algoritm complex in subprograme mai mici. Acest lucru aduce claritate, usurinta in depanare si posibilitatea de a refolosi codul.

Sintaxa de baza in Python:

- **Antetul** foloseste cuvantul cheie **def**, urmat de numele functiei si **parametrii** (datele de intrare).
- **Corpul** contine instructiunile, indentate corespunzator.
- **Returnarea rezultatului** se face prin instructiunea **return**.
- **Variabilele locale** sunt definite in interiorul subprogramului si exista doar acolo. **Variabilele globale** sunt definite in programul principal.

Functii predefinite utile:

Operatie	Funcție Python	Descriere / Exemplu
Valoare absoluta	abs(x)	Elimina semnul negativ: abs(-5) devine 5
Rotunjire matematica	round(x, n)	Rotunjeste x la n zecimale: round(3.75, 1) -> 3.8
Conversie la intreg	int(x)	Trunchiaza zecimalele (nu rotunjeste): int(3.9) -> 3
Partea intreaga (inferioara)	math.floor(x)	Cel mai mare intreg <= x: math.floor(3.9) -> 3
Partea intreaga (superioara)	math.ceil(x)	Cel mai mic intreg >= x: math.ceil(3.1) -> 4
Radacina patrata	math.sqrt(x)	Calculeaza √ x: math.sqrt(16) -> 4.0

2. Exemplu rezolvat

A. Eficienta subprogramelor: Calculul cumparaturilor Sa analizam cum calculam costul total pentru mai multe cosuri de cumparaturi.

Varianta fara subprograme (repetitiva si greu de citit):

```
# Cos 1

pret_baza_1 = 100

taxa_1 = pret_baza_1 * 0.19

total_1 = pret_baza_1 + taxa_1

print("Total 1:", total_1)


# Cos 2

pret_baza_2 = 50

taxa_2 = pret_baza_2 * 0.19

total_2 = pret_baza_2 + taxa_2

print("Total 2:", total_2)
```

Varianta cu subprograme (modulara si reutilizabila):

```
Python

def calculeaza_total(pret_baza): # pret_baza
    este parametru local

    taxa = pret_baza * 0.19    # taxa este
    variabila locala

    return pret_baza + taxa


print("Total 1:", calculeaza_total(100)) #
    Apelul functiei

print("Total 2:", calculeaza_total(50))
```

3. Exerciții diversificate

A. Completează spațiile libere

1. Cuvântul cheie utilizat în Python pentru a defini un subprogram este _____.

2. Datele care sunt transmise unui subprogram in momentul apelului se numesc _____.
3. Daca dorim ca un subprogram sa trimita o valoare inapoi in programul principal, folosim instructiunea _____.
4. Functia `int(4.8)` va returna valoarea _____, in timp ce `round(4.8)` va returna _____.

B. Adevarat sau fals

1. O variabila declarata in interiorul unui subprogram (variabila locala) poate fi accesata direct din programul principal. (A / F)
2. Functia `math.floor()` rotunjeste intotdeauna un numar in jos, catre cel mai apropiat intreg. (A / F)
3. Daca un subprogram nu are parametri, el nu trebuie sa aiba paranteze la definire si la apel. (A / F)
4. Functiile `min()` si `max()` pot fi aplicate pe o lista eterogena (ex: `min([3, "mere", 5])`) fara a genera o eroare. (A / F)

C. Exerciții de lucru

1. Conversie temperatura

Completeaza codul de mai jos pentru un subprogram care primeste ca parametru o temperatura in grade Celsius, o converteste in Fahrenheit (formula: $F = C \times 1.8 + 32$) si returneaza valoarea rotunjita la un intreg.

`def conversie_celsius_fahrenheit(celsius):`

`f = celsius * _____ + 32`

`return _____(f)`

`temp_c = 25.4`

`print("Temperatura in F este:", _____(temp_c))`

2. Aplicatie practica: Analiza datelor meteo

Scrie un subprogram numit `raport_meteo(temperaturi)` care primeste o lista de temperaturi. Subprogramul trebuie sa returneze un mesaj (tip string) care sa contina temperatura maxima si temperatura minima folosind functiile predefinite adecvate. Ruleaza algoritmul pentru cazuri limita (ex: o lista cu o singura temperatura).

Barem orientativ / indicatii

- **A:** 1. `def`; 2. parametri (sau argumente); 3. `return`; 4. 4 si 5.
- **B:** 1-F (domeniul de vizibilitate este strict local); 2-A; 3-F (parantezele sunt obligatorii chiar si goale `()`); 4-F (genereaza eroare `TypeError` deoarece nu poate compara numere cu siruri de caractere).
- **C1.** 1.8, `round`, `conversie_celsius_fahrenheit`.
- **C2. Indicatii:** Folosirea functiilor `max(temperaturi)` si `min(temperaturi)`. Daca lista are un singur element, `min` si `max` vor fi egale.